Projet data : prévoir la consommation électrique française

Oct 22, 2026 · @m

Contexte et objectif

Ta mission : construire un modèle qui prédit la consommation électrique française du lendemain, heure par heure, à partir de l'historique et de la météo, puis le rendre utilisable dans une petite application web.

Le gestionnaire du réseau (RTE) doit en permanence équilibrer production et consommation. Une bonne prévision permet de planifier les centrales, d'éviter les coupures et de limiter les coûts. La consommation dépend fortement de la température (chauffage électrique en hiver), de l'heure, du jour de la semaine et des jours fériés.

À la fin du projet, tu dois pouvoir répondre à trois questions :

1. Quels facteurs expliquent le mieux la consommation électrique en France ?
2. Avec quelle précision peut-on la prévoir 24 heures à l'avance ?
3. Ton modèle fait-il mieux qu'une méthode naïve (par exemple « même heure, une semaine avant ») ?

Durée indicative : 4 à 6 semaines à raison de 5 à 8 heures par semaine.

Compétences travaillées : collecte et fusion de données, nettoyage, analyse exploratoire, séries temporelles, machine learning, évaluation rigoureuse, déploiement d'une démo, Git.

Livrables attendus

Tout doit être dans un dépôt GitHub public, compréhensible par un recruteur en 2 minutes.

- [ ] **Un dépôt GitHub propre** : code organisé en dossiers (`data/` , `notebooks/` , `src/` , `app/`), un `requirements.txt` ou `pyproject.toml` , un `.gitignore` , des commits réguliers avec des messages clairs.
- [ ] **Un notebook d'analyse exploratoire** avec des graphiques commentés.
- [ ] **Un code d'entraînement reproductible** (des scripts Python dans `src/` , pas seulement des notebooks).
- [ ] **Un tableau de résultats** comparant au moins trois approches avec la même métrique.
- [ ] **Une application Streamlit déployée en ligne** qui affiche la prévision et la compare à la réalité.

- [ ] **Un README soigné** : problème, données, démarche, résultats chiffrés, capture d'écran de l'application, lien vers la démo, limites et pistes d'amélioration.

Données et outils

Tu croiseras deux sources gratuites, plus un calendrier des jours fériés.

Source	Ce qu'elle contient	Accès
éCO2mix national consolidé et définitif (RTE)	Consommation et production électriques françaises, au pas infra-horaire, depuis 2012	Export CSV depuis le portail, ou API (quota de 50 000 appels par mois et par utilisateur)
Open-Meteo Historical Weather API	Météo horaire passée (température, vent, nuages, ensoleillement) pour n'importe quelle coordonnée	API gratuite pour un usage non commercial, sans clé : on donne une latitude, une longitude, une date de début et de fin
Bibliothèque Python `holidays`	Jours fériés français	`pip install holidays`

Une difficulté volontaire : la consommation est nationale, la météo est locale. À toi de construire une « température France », par exemple une moyenne pondérée par la population de 8 à 10 grandes villes (Paris, Lyon, Marseille, Lille, Toulouse, Bordeaux, Nantes, Strasbourg…). Justifie ton choix dans le README.

Outils conseillés :

- Python 3, avec pandas (ou Polars), NumPy, Matplotlib ou Plotly
- scikit-learn pour les premiers modèles, puis LightGBM ou XGBoost
- Jupyter pour l'exploration, de vrais scripts `.py` pour le reste
- Streamlit pour l'application, déployée gratuitement sur Streamlit Community Cloud
- Git et GitHub dès le premier jour
- Ton PC Linux comme machine principale, avec un environnement virtuel (`venv` ou `uv`)

Période conseillée : 2015 à aujourd'hui. Les années 2020 (confinements) et 2022 (crise de l'énergie et sobriété) sont atypiques : repère-les et décide comment les traiter.

Étapes du projet

Suis les étapes dans l'ordre. Chacune se termine par un commit et une case à cocher. Les questions guides ne demandent pas de réponse écrite, mais ton travail doit permettre d'y répondre.

Étape 1 — Mise en place (semaine 1)

Consigne : crée le dépôt GitHub, l'arborescence, l'environnement virtuel et un premier README qui décrit le problème en 5 lignes.

- [ ] Dépôt créé avec `.gitignore` (exclure `data/raw/` et les gros fichiers)
- [ ] Environnement virtuel et fichier de dépendances
- [ ] README initial

Étape 2 — Collecte des données (semaine 1)

Consigne : écris un script `src/download.py` qui télécharge la consommation éCO2mix et la météo horaire des villes choisies, et les enregistre dans `data/raw/`. Le script doit pouvoir être relancé sans tout retélécharger.

Questions guides :

- Quel est le pas de temps de chaque source ? Dans quel fuseau horaire sont les dates ?
- Comment gères-tu le quota de l'API (téléchargement par année, mise en cache) ?

- [ ] Script de téléchargement fonctionnel et reproductible

Étape 3 — Nettoyage et fusion (semaine 2)

Consigne : écris `src/prepare.py` qui produit une seule table horaire propre : une ligne par heure, la consommation en MW, la température France, et les autres variables météo retenues. Sauvegarde-la au format Parquet dans `data/processed/`.

Questions guides :

- Comment gères-tu les changements d'heure (l'heure qui manque au printemps, l'heure en double en automne) ?
- Y a-t-il des valeurs manquantes ou aberrantes ? Que décides-tu d'en faire, et pourquoi ?
- Comment vérifies-tu que la fusion n'a pas décalé les données d'une heure ?

- [ ] Table horaire propre, sans doublons, avec des tests simples (nombre de lignes, valeurs manquantes)

Étape 4 — Analyse exploratoire (semaine 2 à 3)

Consigne : dans `notebooks/01_exploration.ipynb`, produis au moins 6 graphiques commentés qui répondent à la question « qu'est-ce qui fait varier la consommation ? ».

Graphiques attendus au minimum :

1. La consommation sur toute la période (tendance et saisonnalité)
2. Le profil moyen d'une journée, en semaine et le week-end
3. Le profil moyen par jour de la semaine
4. La consommation en fonction de la température (nuage de points)
5. L'effet des jours fériés et des vacances d'été
6. Une comparaison des années atypiques (2020, 2022) avec les autres

Questions guides :

- La relation consommation–température est-elle linéaire ? À partir de quelle température le chauffage se déclenche-t-il ?
- Quelle part de la variation vient du calendrier, et quelle part de la météo ?

- [ ] Notebook avec graphiques titrés, axes en unités, et une phrase de conclusion sous chacun

Étape 5 — Variables explicatives et découpage (semaine 3)

Consigne : écris `src/features.py` qui construit les variables du modèle, puis découpe les données **dans l'ordre chronologique** : par exemple entraînement jusqu'à fin 2023, validation sur 2024, test sur 2025 et après.

Variables à envisager :

- Calendrier : heure, jour de la semaine, mois, jour férié, veille de férié, vacances
- Météo : température, température lissée sur 24 ou 48 h (les bâtiments ont de l'inertie), « degrés de chauffage » (écart à un seuil)
- Historique : consommation de la même heure il y a 1 jour et 7 jours

Règle d'or : pour prévoir demain, tu n'as le droit qu'à des informations connues aujourd'hui. La consommation d'il y a 1 heure n'est pas disponible pour prévoir 24 h à l'avance.

Questions guides :

- Pourquoi un découpage aléatoire (`train_test_split` classique) serait-il une erreur ici ?
- En situation réelle, tu utiliserais la température *prévue* pour demain, pas la température observée. Quel effet cela a-t-il sur la précision annoncée ?

- [ ] Fonction de construction des variables, sans fuite d'information, avec découpage chronologique

Étape 6 — Modélisation et évaluation (semaines 3 à 5)

Consigne : compare au moins trois approches sur le jeu de validation, choisis la meilleure, puis évalue-la **une seule fois** sur le jeu de test.

1. **Référence naïve :** la consommation de la même heure, 7 jours avant. Tout modèle doit la battre.
2. **Régression linéaire** avec tes variables (ou régression Ridge).
3. **Gradient boosting** (LightGBM ou XGBoost), avec un réglage simple des hyperparamètres.

Métriques : MAE en MW et MAPE en %. Présente-les dans un tableau, une ligne par modèle.

Questions guides :

- Quelles variables comptent le plus pour le meilleur modèle (importance des variables) ?
- Quand ton modèle se trompe-t-il le plus : quelles heures, quels jours, quelles saisons ? Montre-le avec un graphique des erreurs.
- Le gain par rapport à la référence naïve justifie-t-il la complexité ajoutée ?

- [ ] Tableau de résultats sur validation, résultat final sur test, analyse des erreurs
- [ ] Modèle final sauvegardé (`models/`), ré-entraînable avec une seule commande

Étape 7 — Application et présentation (semaines 5 à 6)

Consigne : construis une application Streamlit dans `app/` et déploie-la en ligne. Elle doit au minimum :

- permettre de choisir une date dans la période de test ;
- afficher la courbe prévue et la courbe réelle sur 24 h ;
- afficher l'erreur du jour ;
- expliquer en quelques lignes ce que fait le modèle.

Termine par le README final (voir les livrables) et un court résumé du projet pour ton CV et LinkedIn, en 3 lignes avec un résultat chiffré.

- [ ] Application déployée et lien dans le README
- [ ] README final avec résultats et captures
- [ ] Résumé de 3 lignes pour le CV

Critères de réussite

Utilise cette grille pour t'auto-évaluer avant de mettre le projet sur ton CV. Un projet « prêt pour un recruteur » coche tout le niveau attendu.

Critère	Niveau minimum	Niveau attendu
Reproductibilité	Le code tourne sur ta machine	Un inconnu clone le dépôt et refait tout avec les commandes du README
Données	Les deux sources sont fusionnées	Changements d'heure, valeurs manquantes et années atypiques traités et justifiés
Rigueur	Un découpage train / test	Découpage chronologique, aucune fuite d'information, test utilisé une seule fois
Modélisation	Un modèle qui fonctionne	Trois approches comparées, référence naïve nettement battue, erreurs analysées
Communication	Un README qui décrit le projet	Résultats chiffrés, graphiques, limites honnêtes, lien vers une démo en ligne
Qualité du code	Des notebooks lisibles	Scripts dans `src/`, fonctions documentées, quelques tests, historique Git propre

Le test final : peux-tu présenter le projet en 2 minutes à l'oral, avec le problème, ta démarche, ton résultat chiffré et une difficulté que tu as surmontée ?

Pour aller plus loin

Une fois le projet terminé, choisis une ou deux de ces extensions. Chacune ajoute une compétence que les recruteurs remarquent.

- **Se comparer aux professionnels :** les fichiers éCO2mix contiennent normalement les prévisions de RTE (vérifie les colonnes de prévision J-1). Compare ton erreur à la leur.
- **Prévision en conditions réelles :** utilise l'API de prévision météo d'Open-Meteo pour prédire le vrai lendemain, et affiche-le dans l'application.
- **Intervalles de prévision :** prédis une fourchette plutôt qu'une valeur (régression quantile avec LightGBM).
- **Deep learning :** essaie un modèle séquentiel (LSTM) ou un modèle de séries temporelles moderne, et regarde s'il bat vraiment le gradient boosting.

- **MLOps :** suivi des expériences avec MLflow, API FastAPI, conteneur Docker, tests automatiques avec GitHub Actions.
- **Pont vers l'embarqué :** plus tard, un ESP32 avec un capteur de courant peut mesurer la consommation d'un appareil chez toi, et tu réutilises la même démarche de prévision.

Pièges fréquents et conseils

- **La fuite d'information** est l'erreur n°1 en séries temporelles : un découpage aléatoire ou une variable connue trop tard donne des résultats excellents… et faux. Si ton modèle semble trop bon, cherche d'abord une fuite.
- **Les fuseaux horaires** : travaille en UTC en interne, et ne convertis en heure de Paris que pour l'affichage.
- **Ne pas commiter les données brutes** : elles sont lourdes. Le script de téléchargement suffit à les retrouver.
- **Rester bloqué sur le modèle** : un modèle simple bien évalué et bien présenté vaut plus qu'un modèle complexe mal expliqué. Termine d'abord une version complète, puis améliore.
- **Commiter souvent** : un historique Git régulier montre ta démarche aux recruteurs.
- **Noter au fur et à mesure** : garde un fichier `JOURNAL.md` avec tes choix et tes difficultés. Il t'aidera énormément pour le README et pour les entretiens.
- **Demander de l'aide intelligemment** : tu peux me poser des questions à chaque étape, mais essaie d'abord seul. C'est en butant sur les problèmes que tu auras des choses à raconter en entretien.